



Ayudantía 2

No Determinismo

Pregunta 1

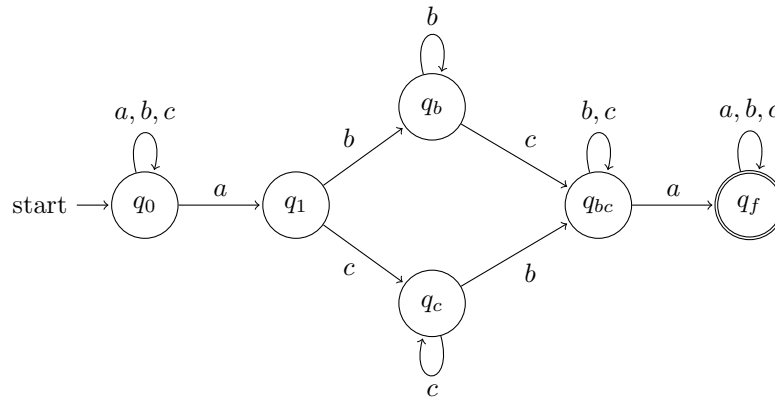
Dado $\Sigma = \{a, b, c\}$, construya un NFA para cada uno de los siguientes lenguajes:

1. $L_1 = \{xayaz \in \Sigma^* \mid y \in \{b, c\}^*, \text{ tanto } b \text{ como } c \text{ aparecen en } y\}$.
2. $L_2 = \{w = uv \mid u \text{ no contiene ninguna } a \text{ y } v \text{ no contiene ninguna } c\}$.

Solución

1. Para reconocer L_1 , el autómata comienza leyendo libremente la palabra. Cada vez que lee una letra a , puede decidir de manera no determinista que esta será la primera de las dos apariciones relevantes de a .

Una vez tomada esta decisión, se debe verificar que antes de la siguiente aparición de a aparezcan tanto una letra b como una letra c .



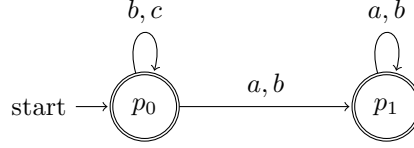
Al leer una letra a , el NFA puede permanecer en q_0 o moverse a q_1 , adivinando que esa aparición de a es la primera relevante de $xayaz$. Luego los demás estados se encargan de que ocurra una b y una c antes de la siguiente a .

Desde q_1 , q_b y q_c no existen transiciones con la letra a , por lo tanto, si aparece una nueva a antes de haber observado tanto una b como una c , esa ejecución no acepta.

2. Para reconocer L_2 , el autómata comienza suponiendo que se encuentra leyendo la parte u , la cual no contiene ninguna letra a , en esta etapa solo puede leer b y c .

En algún momento, el NFA puede decidir de manera no determinista que comenzó la parte v , esto ocurre cuando se lee una b que es el único carácter que comparten u y v . Desde ese momento puede leer a y b , pero no puede volver a leer una letra c .

Un NFA que reconoce L_2 es:



Pregunta 2

Sea Σ un alfabeto y sea $L \subseteq \Sigma^*$ un lenguaje regular. Definimos

$$\text{Del}(L) = \{xy \in \Sigma^* \mid \exists z \in \Sigma^* \text{ tal que } xzy \in L\}.$$

Es decir, $\text{Del}(L)$ contiene todas las palabras que pueden obtenerse eliminando un bloque contiguo de símbolos de alguna palabra de L .

Demuestre que $\text{Del}(L)$ es regular.

Solución

Como L es regular, existe un DFA $A = (Q, \Sigma, \delta, q_0, F)$ tal que $L(A) = L$. Para cada estado $q \in Q$, definimos

$$R(q) = \{r \in Q \mid \text{existe un camino desde } q \text{ hasta } r\}$$

. Permitimos que este camino tenga largo 0, por lo que siempre se cumple que $q \in R(q)$.

La idea del automata es que, después de leer la parte x , el NFA puede decidir de manera no determinista cuál fue el estado alcanzado por A después de leer el bloque z que fue eliminado, por eso, necesitamos poder saltar a cualquier estado alcanzable al terminar de leer x .

Construimos el NFA $N = (Q', \Sigma, \Delta, (q_0, 0), F')$, donde:

- $Q' = Q \times \{0, 1\}$
- Un estado $(q, 0)$ representa que el autómata todavía no ha hecho la eliminación y que la ejecución de A se encuentra en q .
- Un estado $(q, 1)$ representa que el autómata ya hizo la eliminación y que la ejecución se encuentra en q .
- Para todo $q \in Q$ y $a \in \Sigma$, definimos

$$\begin{aligned} \Delta((q, 0), a) &= \{(\delta(q, a), 0)\} \cup \{(\delta(r, a), 1) \mid r \in R(q)\}, \\ \Delta((q, 1), a) &= \{(\delta(q, a), 1)\}. \end{aligned}$$

La primera posibilidad desde $(q, 0)$ corresponde a considerar que la letra a todavía pertenece a x , por lo tanto seguimos las transiciones normales del autómata. En la segunda posibilidad, el NFA adivina que x terminó, escoge algún estado $r \in R(q)$ y considera que el camino desde q hasta r corresponde al bloque z que fue eliminado, de esta forma, la letra a que se está leyendo pasa entonces a ser la primera letra de y .

La primera posibilidad desde $(q, 0)$ corresponde a considerar que la letra a todavía pertenece a x , por lo tanto seguimos las transiciones normales del autómata. En la segunda posibilidad, el NFA adivina que x terminó, escoge algún estado $r \in R(q)$ y considera que el camino desde q hasta r corresponde al bloque z que fue eliminado, de esta forma, la letra a que se está leyendo pasa entonces a ser la primera letra de y .

- Los estados finales son

$$F' = \{(q, 1) \mid q \in F\} \cup \{(q, 0) \mid R(q) \cap F \neq \emptyset\}.$$

Si en algún estado q , tenemos la posibilidad de saltar a un final en $R(q)$ entonces q debe ser final, ya que esto corresponde al caso donde $y = \varepsilon$.

Demostraremos que $L(N) = \text{Del}(L)$ mediante doble contención.

$$\text{Del}(L) \subseteq L(N)$$

Sea $w \in \text{Del}(L)$. Por definición, existen palabras $x, z, y \in \Sigma^*$ tales que $w = xy$ y $xzy \in L$.

Escribamos $x = a_1 \cdots a_k$, $z = b_1 \cdots b_m$ e $y = c_1 \cdots c_\ell$. Como $xzy \in L = L(A)$, existe una ejecución de aceptación de A de la forma:

$$\begin{aligned} \rho : \quad q_0 &\xrightarrow{a_1} q_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} q_k \xrightarrow{b_1} r_1 \xrightarrow{b_2} \cdots \xrightarrow{b_m} r_m \\ &\xrightarrow{c_1} s_1 \xrightarrow{c_2} \cdots \xrightarrow{c_\ell} s_\ell, \end{aligned}$$

donde el último estado de la ejecución pertenece a F . Si $z = \varepsilon$, entendemos que $r_m = q_k$.

Como existe un camino desde q_k hasta r_m , se cumple que $r_m \in R(q_k)$. Supongamos primero que $y \neq \varepsilon$. El NFA puede leer x permaneciendo en la primera copia y siguiendo exactamente los mismos estados que A , luego cuando termina x y lee c_1 , puede escoger de manera no determinista el estado $r_m \in R(q_k)$ y pasar a la segunda copia. Por lo tanto, existe en N la ejecución

$$\begin{aligned} \rho' : \quad (q_0, 0) &\xrightarrow{a_1} (q_1, 0) \xrightarrow{a_2} \cdots \xrightarrow{a_k} (q_k, 0) \\ &\xrightarrow{c_1} (s_1, 1) \xrightarrow{c_2} \cdots \xrightarrow{c_\ell} (s_\ell, 1). \end{aligned}$$

El último estado pertenece a F' porque $s_\ell \in F$. Por lo tanto, ρ' es una ejecución de aceptación de N sobre $w = xy$, y tenemos que $w \in L(N)$.

Supongamos ahora que $y = \varepsilon$. En este caso $w = x$, y la ejecución de A sobre xz termina en algún estado final $r_m \in F$. Como $r_m \in R(q_k)$, tenemos $R(q_k) \cap F \neq \emptyset$. Por definición de F' , el estado $(q_k, 0)$ es final.

Luego,

$$(q_0, 0) \xrightarrow{a_1} (q_1, 0) \xrightarrow{a_2} \cdots \xrightarrow{a_k} (q_k, 0)$$

es una ejecución de aceptación de N sobre w .

En ambos casos $w \in L(N)$, por lo que $\text{Del}(L) \subseteq L(N)$.

$$L(N) \subseteq \text{Del}(L)$$

Sea $w = d_1 \cdots d_n \in L(N)$. Como N acepta w , existe una ejecución de aceptación de N sobre esta palabra.

Tenemos dos casos dependiendo de si esta ejecución pasa o no desde un estado de la forma $(q, 0)$ a un estado de la forma $(r, 1)$.

Caso 1: la ejecución pasa a la segunda copia. Una vez que una ejecución entra a la segunda copia, nunca puede volver a la primera. Por lo tanto, existe una posición k , con $0 \leq k < n$, tal que la ejecución tiene la forma

$$(q_0, 0) \xrightarrow{d_1} (q_1, 0) \xrightarrow{d_2} \dots \xrightarrow{d_k} (q_k, 0) \\ \xrightarrow{d_{k+1}} (p_1, 1) \xrightarrow{d_{k+2}} \dots \xrightarrow{d_n} (p_{n-k}, 1),$$

donde $p_{n-k} \in F$.

Por definición de las transiciones entre ambas copias, existe algún $r \in R(q_k)$ tal que $\delta(r, d_{k+1}) = p_1$.

Como $r \in R(q_k)$, existe un camino en A desde q_k hasta r . Sean $b_1, \dots, b_m$ los caracteres de las transiciones de dicho camino y definamos $z = b_1 \dots b_m$. Tomemos además $x = d_1 \dots d_k$ e $y = d_{k+1} \dots d_n$.

Juntando los tres caminos obtenemos una ejecución de A :

$$q_0 \xrightarrow{x} q_k \xrightarrow{z} r \xrightarrow{y} p_{n-k}.$$

Como $p_{n-k} \in F$, esta es una ejecución de aceptación de A sobre xyz . Por lo tanto, $xyz \in L$. Además, por construcción $w = xy$, y entonces $w \in \text{Del}(L)$.

Caso 2: la ejecución nunca pasa a la segunda copia. La ejecución tiene entonces la forma

$$(q_0, 0) \xrightarrow{d_1} (q_1, 0) \xrightarrow{d_2} \dots \xrightarrow{d_n} (q_n, 0).$$

Como es una ejecución de aceptación, $(q_n, 0) \in F'$. Por definición de F' , se cumple que $R(q_n) \cap F \neq \emptyset$ y por lo tanto, existe un estado $f \in F$ alcanzable desde q_n .

Sea z la palabra formada por los caracteres de algún camino desde q_n hasta f . Las transiciones de la primera copia, seguidas de este camino forman una ejecución de A

$$q_0 \xrightarrow{w} q_n \xrightarrow{z} f.$$

Como $f \in F$, tenemos $wz \in L$. Tomando $x = w$ e $y = \varepsilon$, obtenemos $xyz = wz \in L$, por lo que $w \in \text{Del}(L)$.

Finalmente, $L(N) = \text{Del}(L)$. Como N se vió en clase, para todo NFA existe un DFA equivalente, por lo tanto concluimos que $\text{Del}(L)$ es regular.

Pregunta 3

Una *rotación* de una palabra w se obtiene separando w en dos partes $w = xy$ e intercambiando su orden. Dado un lenguaje $L \subseteq \Sigma^*$, definimos

$$\text{Rot}(L) = \{yx \mid x, y \in \Sigma^* \text{ y } xy \in L\}.$$

Demuestre que si L es regular, entonces $\text{Rot}(L)$ es regular.

Solución

Como L es regular, existe un DFA $A = (Q, \Sigma, \delta, q_0, F)$ tal que $L(A) = L$.

Supongamos que una palabra xy pertenece a L . Si después de leer x el DFA se encuentra en un estado r , entonces una ejecución de aceptación de A tiene la forma

$$q_0 \xrightarrow{x} r \xrightarrow{y} f, \quad f \in F.$$

Para reconocer la rotación yx , el problema es que el autómata recibe primero y , pero para simular correctamente esa parte debería conocer el estado r al que llegará A después de leer x . Utilizaremos el no determinismo para adivinar este estado y luego verificar que la elección fue correcta.

Para cada $r \in Q$, construiremos dos copias de los estados de A :

$$A_r = \{q_r \mid q \in Q\} \quad \text{y} \quad A^r = \{q^r \mid q \in Q\}.$$

La copia A_r se utiliza para leer y . En ella suponemos que r es el estado alcanzado por A después de leer x , por lo que r_r será un estado inicial del NFA.

La copia A^r se utiliza posteriormente para leer x . En ella simulamos A comenzando desde q_0 , y la ejecución será aceptada solamente si termina en el estado r que fue adivinado.

Por ejemplo, si $w = yx$, con $y = b_0 \cdots b_n$ y $x = a_0 \cdots a_m$, una ejecución de aceptación tendrá la forma

$$r_r \xrightarrow{b_0} \cdots \xrightarrow{b_n} f_r \xrightarrow{a_0} q^r \xrightarrow{a_1} \cdots \xrightarrow{a_m} r^r, \quad f \in F,$$

donde $q = \delta(q_0, a_0)$.

A partir de esta ejecución podemos reconstruir en A el camino

$$q_0 \xrightarrow{a_0} q \xrightarrow{a_1} \cdots \xrightarrow{a_m} r \xrightarrow{b_0} \cdots \xrightarrow{b_n} f, \quad f \in F.$$

Es decir, obtenemos una ejecución de aceptación $q_0 \xrightarrow{x} r \xrightarrow{y} f$ sobre xy . Esta es precisamente la idea que formalizaremos, para ello construimos un NFA

$$N = (Q', \Sigma, \Delta, I, F')$$

de la siguiente forma.

El conjunto de estados corresponde a las dos copias descritas anteriormente:

$$Q' = \{q_r \mid q, r \in Q\} \cup \{q^r \mid q, r \in Q\}.$$

Como el NFA debe adivinar el estado r alcanzado después de leer x , permitimos comenzar en r_r para cualquier $r \in Q$. Por lo tanto,

$$I = \{r_r \mid r \in Q\}.$$

Dentro de una copia A_r , el autómata simula las transiciones de A . Además, si se encuentra en un estado q_r con $q \in F$, puede decidir de manera no determinista que terminó de leer y y que la letra que está leyendo corresponde a la primera letra de x .

Ahora definimos este comportamiento en la función de transición, para todo $a \in \Sigma$, definimos

$$\Delta(q_r, a) = \begin{cases} \{(\delta(q, a))_r, (\delta(q_0, a))^r\}, & \text{si } q \in F, \\ \{(\delta(q, a))_r\}, & \text{si } q \notin F. \end{cases}$$

La primera transición continúa simulando y desde el estado r . La segunda opción comienza a simular x desde q_0 .

Una vez que el autómata se encuentra en la copia A^r , simplemente continúa simulando las transiciones de A . Para todo $q, r \in Q$ y $a \in \Sigma$, definimos

$$\Delta(q^r, a) = \{(\delta(q, a))^r\}.$$

Si la ejecución entra a A^r , deberá terminar precisamente en r^r . Esto verifica que la simulación de x comenzando desde q_0 efectivamente terminó en el estado r que había sido adivinado, esta idea se expresa en el siguiente conjunto de estados finales:

$$F' = \{r^r \mid r \in Q\} \cup \{f_{q_0} \mid f \in F\}.$$

Noten que también se incluye f_{q_0} para todo $f \in F$ esto se debe a que podemos recibir palabras donde $y = \varepsilon$ y $x \in L$, por lo tanto, tenemos que darle la posibilidad a nuestro NFA que ejecute el automata

A directamente. Para lograr esto utilizamos la copia A_{q_0} , donde $(q_0)_{q_0}$ es el estado inicial, ya que esta copia simula directamente a A desde su estado inicial.

Esto también maneja correctamente la palabra vacía. Si $\varepsilon \in L$, entonces $q_0 \in F$, por lo que $(q_0)_{q_0}$ es simultáneamente un estado inicial y final de N .

Ahora hay que demostrar que

$$L(N) = \text{Rot}(L)$$

mediante doble contención.

$$\text{Rot}(L) \subseteq L(N)$$

Sea $w \in \text{Rot}(L)$. Por definición, existen palabras $x, y \in \Sigma^*$ tales que $w = yx$ y $xy \in L$.

Consideremos primero el caso en que $x \neq \varepsilon$ e $y \neq \varepsilon$. Escribamos $x = a_1 \cdots a_k$ e $y = b_1 \cdots b_m$.

Como $xy \in L = L(A)$, existe una ejecución de aceptación de A sobre xy de la forma

$$\begin{aligned} \rho : \quad q_0 &\xrightarrow{a_1} q_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} r \\ &\xrightarrow{b_1} p_1 \xrightarrow{b_2} \cdots \xrightarrow{b_m} f, \end{aligned}$$

donde $f \in F$.

Como $r_r \in I$, el NFA puede comenzar su ejecución en la copia A_r . Las transiciones de esta copia permiten simular la parte $r \xrightarrow{y} f$ de la ejecución anterior:

$$r_r \xrightarrow{b_1} (p_1)_r \xrightarrow{b_2} \cdots \xrightarrow{b_m} f_r.$$

Como $f \in F$, al leer la primera letra a_1 de x , el NFA puede decidir que terminó de leer y y pasar a la copia A^r . A partir de ese momento simula desde q_0 la ejecución de A sobre x . De esta forma existe la ejecución

$$\begin{aligned} \rho' : \quad r_r &\xrightarrow{b_1} (p_1)_r \xrightarrow{b_2} \cdots \xrightarrow{b_m} f_r \\ &\xrightarrow{a_1} (q_1)^r \xrightarrow{a_2} \cdots \xrightarrow{a_k} r^r. \end{aligned}$$

Como $r^r \in F'$, ρ' es una ejecución de aceptación de N sobre $yx = w$. Por lo tanto, $w \in L(N)$.

Supongamos ahora que $x = \varepsilon$ o $y = \varepsilon$. Como $xy \in L$, en cualquiera de estos dos casos se cumple que

$$w = yx = xy \in L.$$

Como $(q_0)_{q_0} \in I$, el NFA puede comenzar en la copia A_{q_0} y simular directamente la ejecución de A sobre w . Como $w \in L$, esta ejecución termina en algún estado $f \in F$, por lo que en N termina en $f_{q_0} \in F'$.

Así, N también acepta w .

En ambos casos $w \in L(N)$. Por lo tanto, $\text{Rot}(L) \subseteq L(N)$.

$$L(N) \subseteq \text{Rot}(L)$$

Sea $w \in L(N)$. Como N acepta w , existe una ejecución de aceptación de N sobre esta palabra.

Tenemos dos posibles formas para una ejecución de aceptación.

Caso 1: la ejecución pasa desde una copia A_r a la copia A^r . Supongamos que la ejecución comienza en r_r , procesa una primera parte de la palabra en A_r y posteriormente pasa a A^r .

Como una vez que la ejecución entra a A^r nunca puede volver a A_r , podemos escribir $w = yx$, donde y corresponde a los símbolos procesados antes del cambio de copia y x corresponde a los símbolos procesados desde el momento en que se realiza dicho cambio.

La ejecución tiene entonces la forma

$$r_r \xrightarrow{y} f_r \xrightarrow{x} r^r, \quad f \in F,$$

donde la primera letra de x es consumida en la transición desde A_r hacia A^r .

Por construcción de las transiciones dentro de A_r , el primer tramo corresponde en A a un camino

$$r \xrightarrow{y} f.$$

Por construcción de las transiciones de A^r , el segundo tramo corresponde en A a un camino

$$q_0 \xrightarrow{x} r,$$

pues la ejecución termina en el estado final r^r .

Juntando ambos caminos obtenemos la ejecución

$$q_0 \xrightarrow{x} r \xrightarrow{y} f, \quad f \in F.$$

Por lo tanto, A acepta xy , de modo que $xy \in L$. Como $w = yx$, por definición tenemos $w \in \text{Rot}(L)$.

Caso 2: la ejecución nunca pasa a una copia A^r . En este caso toda la ejecución permanece en alguna copia A_r .

Como los únicos estados finales de este tipo son f_{q_0} con $f \in F$, necesariamente la ejecución comenzó en la copia A_{q_0} . Por lo tanto, la ejecución tiene la forma

$$(q_0)_{q_0} \xrightarrow{w} f_{q_0}, \quad f \in F.$$

Por construcción de las transiciones de A_{q_0} , esto corresponde en A a una ejecución

$$q_0 \xrightarrow{w} f, \quad f \in F.$$

Por lo tanto, $w \in L$.

En ambos casos $w \in \text{Rot}(L)$. Por lo tanto, $L(N) \subseteq \text{Rot}(L)$.

Finalmente, $L(N) = \text{Rot}(L)$. Como N es un autómata finito, concluimos que $\text{Rot}(L)$ es regular.